UNIT 3

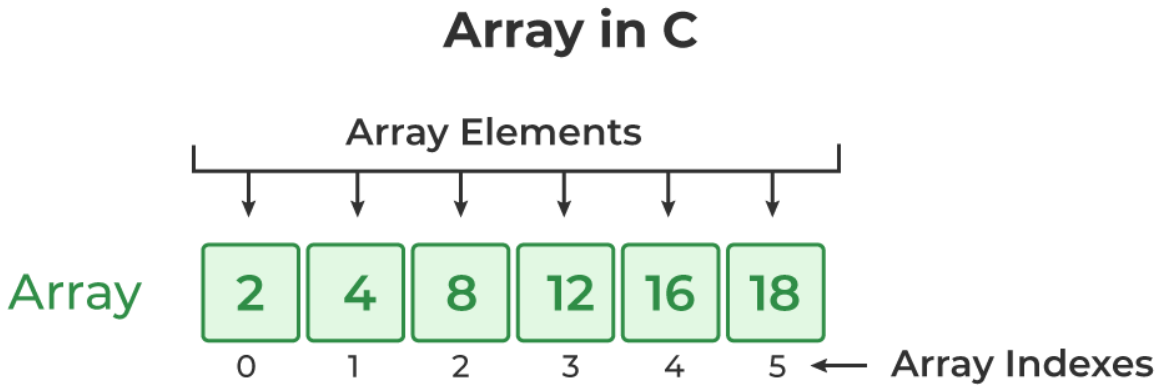
Arrays and structure

3.1 C Arrays

Array in C is one of the most used data structures in C programming. It is a simple and fast way of storing multiple values under a single name. In this chapter, we will study the different aspects of array in C language such as array declaration, definition, initialization, types of arrays, array syntax, advantages and disadvantages, and many more.

What is Array in C?

An array in C is a fixed-size collection of similar data items stored in contiguous memory locations. It can be used to store the collection of primitive data types such as int, char, float, etc., and also derived and user-defined data types such as pointers, structures, etc.



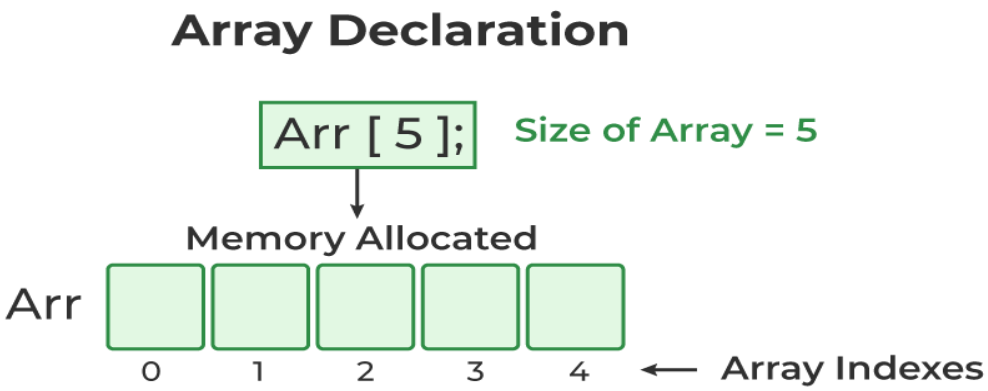
C Array Declaration

In C, we have to declare the array like any other variable before using it. We can declare an array by specifying its name, the type of its elements, and the size of its dimensions. When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

Syntax of Array Declaration

data_type array_name [size];
OR
data_type array_name [*size1*] [*size2*]...[*sizeN*];

where N is the number of dimensions.



C Program to illustrate the array declaration

```
#include <stdio.h>
int main()
{
    // declaring array of integers
    int arr_int[5];
    // declaring array of characters
    char arr_char[5];

    return 0;
}
```

C Array Initialization

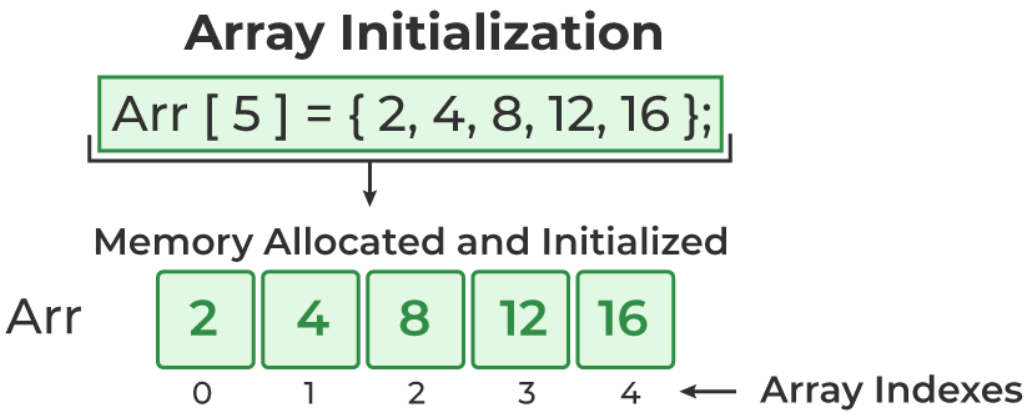
Initialization in C is the process to assign some initial value to the variable. When the array is declared or allocated memory, the elements of the array contain some garbage value. So, we need to initialize the array to some meaningful value. There are multiple ways in which we can initialize an array in C.

1. Array Initialization with Declaration

- In this method, we initialize the array along with its declaration. We use an initializer list to initialize multiple elements of the array. An initializer list is the list of values enclosed within braces **{ }** separated by a comma.

Syntax:

data_type array_name [*size*] = {value1, value2, ... valueN};



2. Array Initialization with Declaration without Size

- If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases. The size of the array in these cases is equal to the number of elements present in the initializer list as the compiler can automatically deduce the size of the array.

Example:

data_type array_name[] = {1,2,3,4,5};

The size of the above arrays is 5 which is automatically deduced by the compiler.

3. Array Initialization after Declaration (Using Loops)

- We initialize the array after the declaration by assigning the initial value to each element individually. We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

Example:

```
for (int i = 0; i < N; i++)
 {
 array_name[i] = value i  ;
 }
```

Example of Array Initialization in C

```
// C Program to demonstrate array initialization
#include <stdio.h>
int main()
{

    // array initialization using initialier list
    int arr[5] = { 10, 20, 30, 40, 50 };

    // array initialization using initializer list without
    // specifying size
    int arr1[] = { 1, 2, 3, 4, 5 };

    // array initialization using for loop
    float arr2[5];
    for (int i = 0; i < 5; i++) {
        arr2[i] = (float)i * 2.1;
    }
    return 0;
}
```

How to use Array in C?

The following program demonstrates how to use an array in the C programming language:

```
// C Program to demonstrate the use of array
#include <stdio.h>

int main()
{
    // array declaration and initialization
    int arr[5] = { 10, 20, 30, 40, 50 };

    // modifying element at index 2
    arr[2] = 100;

    // traversing array using for loop
  printf("Elements in Array: ");
    for (int i = 0; i < 5; i++)
   {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

Elements in Array: 10 20 100 40 50

Types of Array in C

There are two types of arrays based on the number of dimensions it has. They are as follows:

1. One Dimensional Arrays (1D Array)
2. Multidimensional Arrays
    A. Two Dimensional Arrays (2D Array)
    B. Three Dimensional Array (3 D Array)

1. One Dimensional Array in C

The One-dimensional arrays, also known as 1-D arrays in C are those arrays that have only one dimension.

One-dimensional arrays, also known as single arrays, are arrays with only one dimension or a single row.

Syntax of 1D Array in C
data type array_name [size];

Example of 1D Array in C

```
// C Program to illustrate the use of 1D array
#include <stdio.h>

int main()
{

    // 1d array declaration
    int arr[5];

    // 1d array initialization using for loop
    for (int i = 0; i < 5; i++)
     {
        arr[i] = i * i - 2 * i + 1;
    }

    printf("Elements of Array: ");

    // printing 1d array by traversing using for loop
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:
Elements of Array: 1 0 1 4 9

Array of Characters (Strings)

In C, we store the words, i.e., a sequence of characters in the form of an array of characters terminated by a NULL character. These are called strings in C language.

```
// C Program to illustrate strings
#include <stdio.h>

int main()
{

    // creating array of character
    char arr[6] = { 'C', 'W', 'I', 'T', '\0' };

    // printing string
    int i = 0;
    while (arr[i]) {
        printf("%c", arr[i++]);
    }
    return 0;
}
```

Output:

CWIT

2. Multidimensional Array in C

Multi-dimensional Arrays in C are those arrays that have more than one dimension. Some of the popular multidimensional arrays are 2D arrays and 3D arrays. We can declare arrays with more dimensions than 3d arrays but they are avoided as they get very complex and occupy a large amount of space.

A. Two-Dimensional Array in C

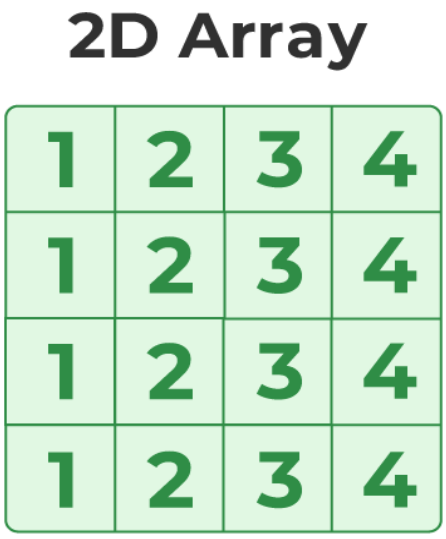
A Two-Dimensional array or 2D array in C is an array that has exactly two dimensions. They can be visualized in the form of rows and columns organized in a two-dimensional plane.

Syntax of 2D Array in C

```
data type array_name[size1] [size2];

    OR

data_type array_name[rows][columns];
```



Example of 2D Array in C

```
// C Program to illustrate 2d array
#include <stdio.h>

int main()
{

    // declaring and initializing 2d array
    int arr[2][3] = { 10, 20, 30, 40, 50, 60 };

  printf("2D Array:\n");
    // printing 2d array
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ",arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output :

2D Array:

10 20 30

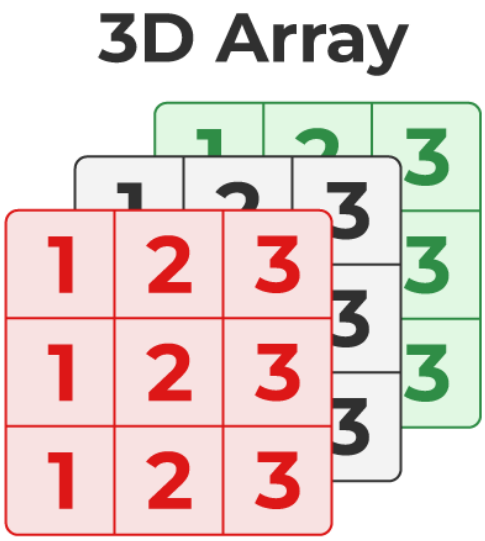
40 50 60

B. Three-Dimensional Array in C

Another popular form of a multi-dimensional array is Three Dimensional Array or 3D Array. A 3D array has exactly three dimensions. It can be visualized as a collection of 2D arrays stacked on top of each other to create the third dimension.

Syntax of 3D Array in C

data type array_name [size1] [size2] [size3];



Example of 3D Array

```
// C Program to illustrate the 3d array
#include <stdio.h>
int main()
{
    // 3D array declaration
    int arr[2][2][2] = { 10, 20, 30, 40, 50, 60 };

    // printing elements
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                printf("%d ", arr[i][j][k]);
            }
            printf("\n");
        }
           }
    return 0;
}
```

Output:

10 20

30 40

50 60

Characteristics of Arrays in C:

1. Fixed Size

The array in C is a fixed-size collection of elements. The size of the array must be known at the compile time and it cannot be changed once it is declared.

2. Homogeneous Collection

We can only store one type of element in an array. There is no restriction on the number of elements but the type of all of these elements must be the same.

3. Indexing in Array

The array index always starts with 0 in C language. It means that the index of the first element of the array will be 0 and the last element will be N – 1.

4. Dimensions of an Array

A dimension of an array is the number of indexes required to refer to an element in the array. It is the number of directions in which you can grow the array size.

5. Contiguous Storage

All the elements in the array are stored continuously one after another in the memory. It is one of the defining properties of the array in C which is also the reason why random access is possible in the array.

6. Random Access

The array in C provides random access to its element i.e we can get to a random element at any index of the array in constant time complexity just by using its index number.

Examples of C array:

1. C Program to Copy All the Elements of One Array to Another Array

```
// C program to copy all the elements of one array to another
#include <stdio.h>

int main()
{

    int a[5] = { 3, 6, 9, 2, 5 }, n = 5;
    int b[n], i;

    // copying elements from one array to another
    for (i = 0; i < n; i++) {
        b[i] = a[i];
    }
    // displaying first array before copy the elements from one array to other
    printf("The first array is :");
    for (i = 0; i < n; i++)
   {
        printf("%d ", a[i]);
    }
    // displaying array after copy the elements from one  array to other
    printf("\nThe second array is :");
    for (i = 0; i < n; i++)
    {
        printf("%d ", b[i]);
    }
    return 0;
}
```

Output:

The first array is: 3 6 9 2 5

The second array is: 3 6 9 2 5

2. Write a C program which will find largest number of array.

```
#include <stdio.h>

int main() {

   int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0};

   int i, largest;

   largest = array[0];

   for(i = 1; i < 10; i++)

 {

      if( largest < array[i] )

         largest = array[i];

   }

   printf("Largest element of array is %d", largest);

   return 0;

}
```

Output:

Largest element of array is 9

3. Write a C program to print 1-D array elements. (Accept input from user)

```
#include <stdio.h>
int main() {
  int values[5];
  printf("Enter 5 integers: ");
  // taking input and storing it in an array
  for(int i = 0; i < 5; i++) {
     scanf("%d", &values[i]);
  }
  printf("Displaying integers: ");
  // printing elements of an array
  for(int i = 0; i < 5; i++) {
     printf("%d\n", values[i]);
  }
  return 0;
}
```

Output:

```
Enter 5 integers: 2 4 6 8 9
Displaying integers: 2
4
6
8
9
```